# Mechanically Governed Generative Storytelling

*Procedural Narrative Middleware as a Bridge Between Game Rules and AI Storytellers*

**Concept paper — working draft 0.1 | October 2026**

## Abstract

Tabletop role-playing games have long used random tables, rules, and prompts to produce situations that a human game master or solo player turns into a story. Large language models (LLMs) offer a new option: they can perform much of that interpretation and narration. Yet an AI storyteller is not a reliable rules engine. It may forget limitations, contradict earlier events, or turn an attempted action into an unauthorized success. This paper describes an architecture developed through practical AI Dungeon scripting experiments: **mechanical rules and persistent state produce a compact, procedurally generated layer of AI-readable instructions, which an AI then turns into prose**. We call that intermediate layer *procedural narrative middleware* and its output *pseudo-prose*. The proposal is deliberately modest. It is an engineering model and a research direction, not a claim of a new scientific field or a solved problem in AI compliance.

## 1. The idea came from a GM-less game

The conceptual starting point was *Across a Thousand Dead Worlds* (ATDW), a science-fiction role-playing game designed for solo and GM-optional play. Its published materials describe extensive tables for generating sites, missions, opponents, and other situations. Such tables do not, by themselves, create a complete narrated scene. They provide facts and prompts that someone must interpret: a location, a hazard, a creature's behavior, or an unexpected encounter.

In a traditional solo session, the player performs two jobs. As a **player**, they decide what their character attempts. As an **interpreter**, they combine dice rolls, procedural results, game rules, and imagination into a coherent fictional world. Even if the tables supply nearly every mechanical ingredient, the player must still connect the ingredients into meaningful events.

While adapting procedural RPG systems to AI Dungeon, we noticed a simple change in this arrangement: **put an AI storyteller between the procedural results and the player, and put a persistent script beneath the tables**. The script remembers and applies rules. The tables generate situation-specific facts. The AI converts those facts into a scene. The human can focus more fully on playing a character.

This change is not merely “use AI to tell a story.” It suggests a division of labor, and an explicit boundary between *what is true in the game* and *how that truth is described*.

## 2. A five-layer story stack

The proposed stack is:

```
MECHANICAL / SCRIPT
        ↓
PROCEDURALLY GENERATED PSEUDO-PROSE
        ↓
AI STORYTELLER
        ↓
NARRATIVE PROSE
        ↓
PLAYER
       ↺ next action returns to the mechanical layer
```

**Layer 1 — Mechanical/script.** JavaScript (or another conventional program) owns authoritative rules and state: registered abilities, inventory, health, positions, dice, hazards, permissions, and outcomes that are actually computable. Procedural tables may be implemented here as data and selection functions. This layer should not derive lasting mechanical facts merely because the narrator happened to say something.

**Layer 2 — Procedural pseudo-prose.** Middleware selects the facts relevant to the present action and translates them into short, AI-readable directions. It can express both permissions and prohibitions, as well as uncertainty or a delegated decision. The output is not polished fiction. It is *instructional material intended to shape fiction*.

**Layer 3 — AI storyteller.** The language model interprets those directions alongside existing story context. It supplies dialogue, sensory detail, social reactions, and connections between procedural facts. Its creative latitude is broad, but it is not supposed to redefine the player's mechanics.

**Layer 4 — Narrative prose.** The player sees the resulting scene, not a stream of internal tables and status flags. A failed spell may be described as a flicker of heat that never reaches its target rather than as an exposed numerical error message.

**Layer 5 — Player.** The player experiences the world, chooses the next action, and sends it back into the loop. That action may itself be natural language and may require interpretation before a script can resolve it.

The stack describes both a **flow of information** and a **hierarchy of mechanical authority**. The distinction matters. The AI may determine an NPC's tone or invent a plausible room detail, but it should not silently grant the player a power absent from the script's registry. Where the script lacks information—such as whether a mysterious NPC has a private mental defense—the architecture must deliberately delegate judgment rather than pretend the script knows the answer.

## 3. The middleware is the critical interface

The most distinctive engineering insight is the intermediate representation. Conventional code and a language model do not naturally consume information in the same form. A script understands structures, numbers, comparisons, and explicit rules. A storyteller works primarily through natural-language instructions, context, and narrative implications.

Consider a character registered with Telekinesis that can move at most 100 pounds. The player attempts to lift a 300-pound stone. The mechanical layer can express the essential result in a few fields:

```
ability: Telekinesis
registered: yes
mass_limit_lb: 100
target_mass_lb: 300
mechanical_result: exceeds_limit
```

Passing these fields alone to an AI does not guarantee that it will obey their implications. Middleware can instead assemble a compact directive:

> *The player may attempt to lift the 300-pound stone, but their registered Telekinesis is limited to 100 pounds. The stone must not rise or shift because of this ability. Describe the effort and its failure naturally; do not increase the player's power.*

This is *pseudo-prose*: grammatically readable language generated from mechanical facts, written for the AI rather than for the audience. It provides **the permitted narrative envelope** without prescribing every sentence of the final narration.

A useful middleware layer should be **selective**, delivering only relevant rules; **state-grounded**, deriving claims from the registry; **explicit about constraints**, including negative permissions; **compact**, respecting limited context space; and **traceable**, so diagnostics can show which ability and constraints were sent. The existing script-to-AI handoff matters more than the raw number of entries in a reference library. A large rules library can remain dormant until an action makes part of it relevant.

## 4. Lessons from working prototypes

Several AI Dungeon scripting projects helped reveal different sides of this architecture.

**Surge Protector: ability boundaries.** A player registers powers and levels in JavaScript. The system attempts to recognize a natural-language action and inject the relevant constraints into the AI's context. In live testing, a 300-pound telekinetic lift failed appropriately under a 100-pound cap; a 12-foot fire-ignition attempt failed against a 10-foot range cap. But a minor-healing ability once repaired a broken bone despite explicit instructions that it could not, and succeeded only in respecting that restriction on retry. The lesson is not that the middleware guarantees obedience. It is that recognition, mechanical permission, AI compliance, and narrative quality must be tested separately.

**MindRift: events and persistent psychological state.** Here, the script tracks quantities such as stress and sanity while the AI portrays their consequences. The middleware must convert numerical conditions and triggered effects into behavior the narrator can express, without turning every mood description into a mechanical event. This is the same translation problem, but the mechanics govern psychology instead of magic.

**Sub Terra and Empire of Echoes: world events.** Action-point rules, procedural rooms, hazards, travel events, and creature-generation tables can be held in state. The middleware presents an AI narrator with only the current facts and restrictions. For example, an illustrative encounter might specify a predatory, armored creature that prefers ambushes in an abandoned facility. Those facts do not determine its exact appearance, dialogue, or dramatic introduction; they bound what the narrator should portray.

These prototypes also exposed the difficult *return path*. Natural-language input must be classified into candidate actions. Sometimes an AI-generated report or narrative outcome must be interpreted back into structured state. Such feedback is useful but fallible. A narrated success should not automatically become an authoritative new rule, and low-confidence outcome audits should not silently rewrite state.

## 5. Why this is promising—and what it cannot yet guarantee

The architecture combines complementary strengths. Ordinary scripts are precise, repeatable, and persistent but poor at improvising prose. Procedural tables provide variety but often need an interpreter. Language models connect fragments into lively scenes but can mishandle constraints. Middleware is the interface that makes these components cooperate without pretending they are interchangeable.

There are significant limitations. **A prompt is not a hard enforcement boundary**: even clear pseudo-prose can be ignored. Natural-language action detection can miss ordinary phrasing or falsely identify an ability. The system may not know key facts, such as an NPC's hidden defenses or an object's weight. Overly verbose guidance can crowd out story context. An output auditor that guesses at consequences can introduce new errors if treated as mechanically authoritative.

For that reason, the architecture should favor explicit authority boundaries, conservative state changes, context-budget management, and honest uncertainty. Where possible, a mechanically determinable outcome should be calculated before narration. Where it is not determinable, the script should state the relevant limits and delegate the unresolved portion clearly. This is a design pattern for *mechanically governed* storytelling, not a claim that all narrative events must become deterministic.

## 6. A path toward a more formal architecture

The next research step would be to specify the contract between the first two layers. One possible middleware interface would accept: **(a)** authoritative state, **(b)** a candidate player action, **(c)** relevant rule profiles, and **(d)** an output budget. It would produce: **(1)** an authorization status or explicit uncertainty, **(2)** a minimal set of applicable constraints, **(3)** an AI-facing pseudo-prose packet, and **(4)** trace data identifying the source of each claim.

The approach could then be evaluated through controlled comparisons: AI narration without middleware, with unstructured rules dumps, and with generated pseudo-prose. Useful measurements would include correct recognition of natural actions, violations of registered limits, unnecessary refusals, continuity errors, packet size, and the player's perceived narrative quality. Representative character builds—a psychic, a shapeshifter, a wizard—would test whether the method generalizes beyond a few carefully tuned examples.

Related research already examines procedural content generation, its combination with language models, and the limits of AI-generated game dialogue. The distinctive *working hypothesis* here is that a structured, state-grounded, procedurally generated natural-language interface can improve the cooperation between deterministic mechanics and a flexible narrator. Establishing how much it helps, and under what conditions, requires further testing.

## Conclusion

GM-less role-playing games demonstrated that tables and rules can generate enough narrative ingredients for a person to imagine a story. Our scripting experiments suggest that a language model can assume much of that interpretive role **if a reliable mechanical layer supplies a carefully prepared diet of facts and constraints**.

The essential idea can be stated simply: **the script decides what is mechanically true; the middleware decides what the AI needs to know about that truth; the AI decides how to tell the story.** The player then acts, and the cycle begins again. That separation—not any particular dice table, prompt, or model—is the architectural insight worth exploring further.

## Sources and context

1. Blackoath Entertainment. *Across a Thousand Dead Worlds*, official game description and GM-less procedural tools. https://blackoathgames.com/across-a-thousand-dead-worlds

2. Farrokhi Maleki, M., and Zhao, R. (2024). “Procedural Content Generation in Games: A Survey with Insights on Emerging LLM Integration.” *Proceedings of AIIDE*, 20(1), 167–178. https://doi.org/10.1609/aiide.v20i1.31877

3. Salmaze, P. H., et al. (2026). “Narrative and dialogue generation for video-games: A systematic mapping.” *Engineering Applications of Artificial Intelligence*, 178, 115041. https://doi.org/10.1016/j.engappai.2026.115041

4. Practical context: iterative, informal AI Dungeon scripting and live playtests of Surge Protector, MindRift, Sub Terra, and Empire of Echoes. These examples are development observations, not controlled scientific experiments.